Ultidock: An Open-Source Docking Pipeline with GPU Acceleration, Automated Cavity Detection, and Molecular Dynamics Integration

Taha Turgut[1]
[1] Department of Physics Engineering, Hacettepe University, Ankara, Türkiye
HIBIT 2025

Scientific Context

Molecular docking predicts how ligands bind to target proteins, guiding early drug discovery. Yet docking alone assumes rigid structures and simplified scoring, often missing solvent and entropic effects.

Molecular dynamics (MD) complements docking by validating top complexes under realistic, flexible, and thermally fluctuating conditions. Together, docking + MD provide a fast and physically grounded pipeline for reliable binding predictions. Modern GPU-accelerated workflows —such as Ultidock— make this hybrid approach feasible at proteome scale, combining high-throughput screening with molecular accuracy.

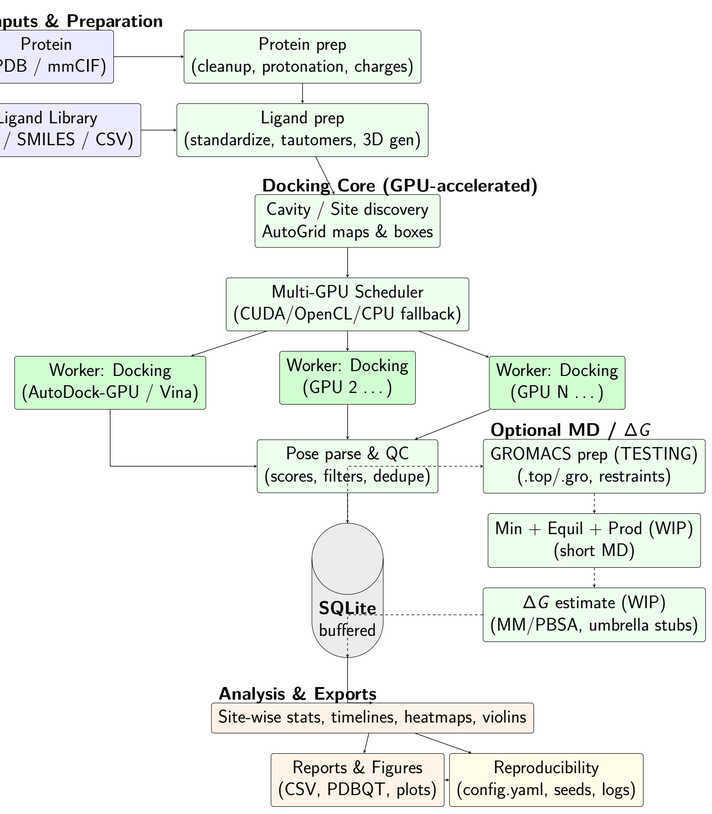
Motivation

High-Throughput Virtual Screening (HTVS) is vital for drug discovery but computationally expensive.

Commercial docking tools are often closed-source, costly, and difficult to automate for large-scale pipelines.

Our Goal: Create Ultidock, an open-source, reproducible, and scalable pipeline that leverages modern multi-GPU hardware.

Ultidock Pipeline



Core Features

Idempotent Setup: Pipeline can be stopped and restarted, reusing cached grids and compiled binaries.

Automated Cavity Detection: A high-precision algorithm automatically finds docking hotspots and sets grid box sizes.

Database-Native: All results are streamed live into an SQLite database for real-time, query-based analysis.

Open-Source Toolchain: Integrates AutoDock-GPU, AutoGrid, and Vina for a fully auditable workflow.

Benchmark Setup

Platform: RunPod Cloud
Hardware: 1× NVIDIA RTX A40 vs. 5× NVIDIA RTX A40
Target: Human Serotonin Transporter (SERT, 5i6x)
Workload: 26 ligands × 64 conformers × 6 binding sites = **9,984 total docking jobs**.

Key Result: 1-GPU vs 5-GPU Speedup

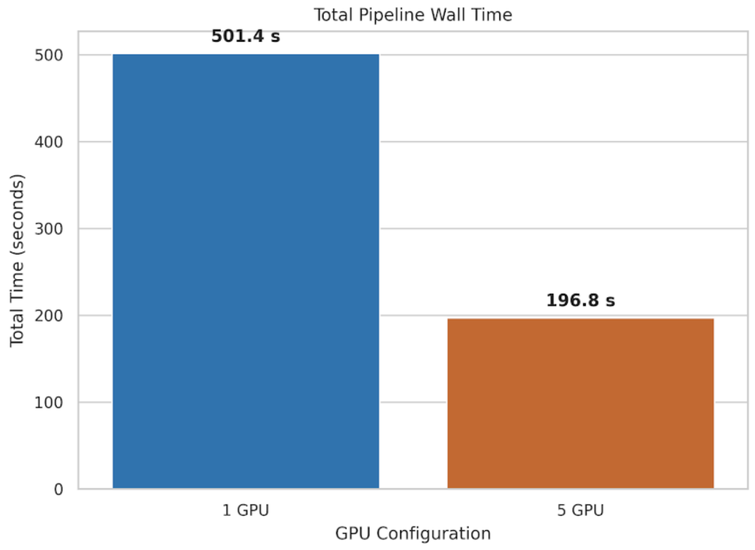


Figure: Scaling from 1 to 5 GPUs reduced the total wall-clock time by **2.55**×, from 8.4 to 3.3 minutes.

Performance Bottleneck: Orchestration

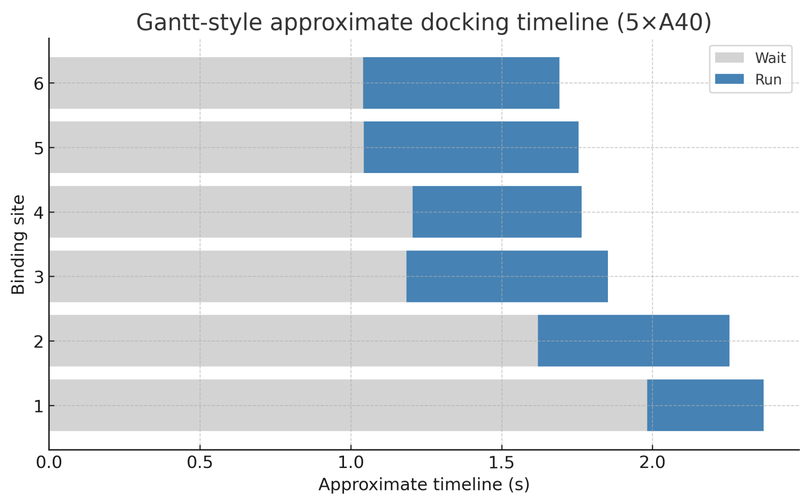


Figure: Gantt timeline (5-GPU run). Jobs (blue) run in parallel but are staggered by a significant "Wait" time (gray).

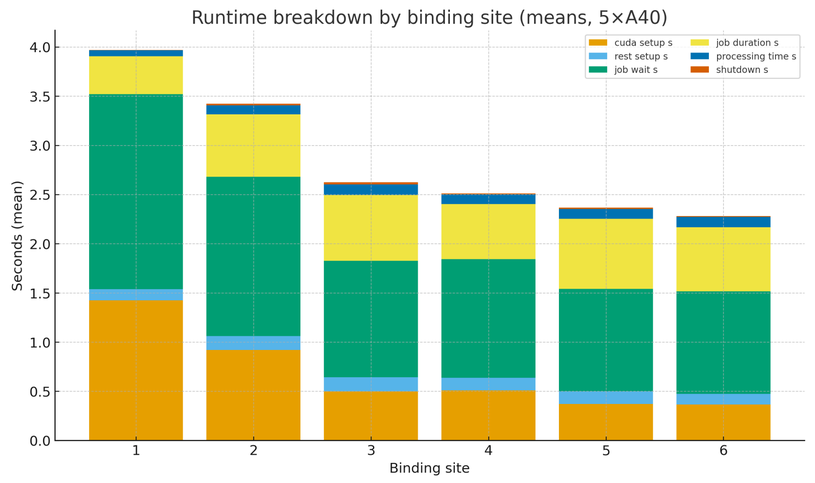


Figure: The 'job_wait_s' (teal)—scheduler wait time—is the largest contributor to total time, not the 'job_duration_s' (yellow) on the GPU.

Validation: Automated Cavity ID

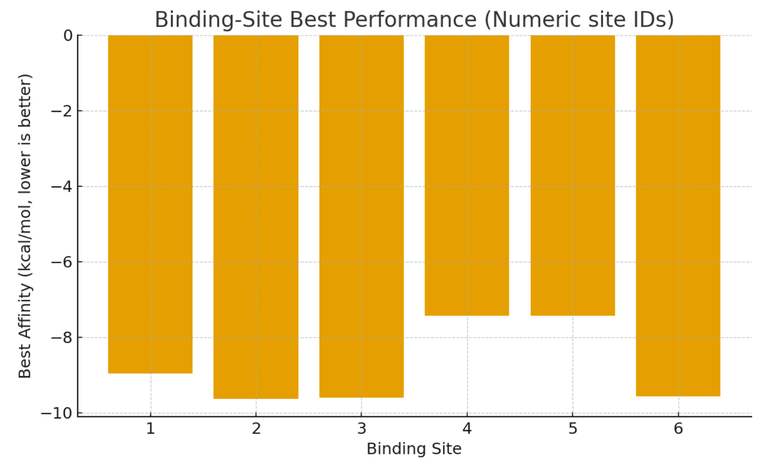


Figure: The Ultidock automated cavity-finding algorithm successfully identified the known orthosteric binding sites of escitalopram (S1, S2, S3, S6), which show strong binding affinity (data from preliminary Vina runs).

Key Findings

Successful Validation: Automated grid algorithm **successfully identified the known orthosteric binding sites** (S2, S3, S6) of escitalopram on SERT.

Significant Speedup: Achieved a **2.55**× reduction in total wall time when scaling from 1 to 5 NVIDIA RTX A40 GPUs.

Bottleneck Identified: Performance is **orchestration-bound, not compute-bound**. The primary bottleneck is scheduler latency ('job_wait_s'), not the GPU calculation.

High Efficiency: The pipeline has a low, stable GPU memory footprint (∼70 MB/job).

Future Directions

Dynamic Job Queue: Replace the static scheduler with a dynamic producer-consumer model to minimize GPU idle time.

Parallelize Setup: Parallelize the per-site AutoGrid setup phase to eliminate the initial staggered start.

Gromacs Integration: Finalize the GROMACS integration for MM/PBSA and umbrella sampling to refine top-scoring poses.

Availability & Acknowledgements

The complete open-source code, configuration files, and analysis scripts are available on GitHub:
https://github.com/taka78/ultidock

Benchmark workloads were executed on the RunPod cloud computing platform. Developed independently at Hacettepe University.

Key References:

Trott, O., & Olson, A. J. (2010). AutoDock Vina: Improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of Computational Chemistry, 31(2)*, 455–461.

Santos-Martins, D., et al. (2021). Accelerating AutoDock4 with GPUs and Gradient-Based Local Search. *Journal of Chemical Theory and Computation, 17(2)*, 1060–1073.